

Sorting Techniques Notes

Bubble • Modified Bubble • Selection • Insertion • Shell • Comb Sort

Quick Comparison Table

Algorithm	Best	Average	Worst	Space	Stable?	Adaptive?	In-Place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Yes
Modified Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Yes
Shell Sort	$O(n \log n)$	depends*	$O(n^2)$	$O(1)$	No	Yes	Yes
Comb Sort	$O(n \log n)$	$O(n^2/2^p)^*$	$O(n^2)$	$O(1)$	No	Yes	Yes

*Shell Sort's average/worst case depends entirely on the gap sequence chosen — plain $n/2$ halving gives $O(n^2)$ worst case; Knuth's sequence $(3k+1)$ improves this to about $O(n^{1.5})$. p in Comb Sort = number of passes.

1. Bubble Sort

Idea: Repeatedly compare adjacent elements and swap if out of order. The largest unsorted element "bubbles up" to the end after each pass.

Visual: One Pass on [5, 1, 4, 2, 8]

5	1	4	2	8
---	---	---	---	---

Step 1 — compare 5 & 1 → out of order → swap

1	5	4	2	8
---	---	---	---	---

Step 2 — compare 5 & 4 → out of order → swap

1	4	5	2	8
---	---	---	---	---

Step 3 — compare 5 & 2 → out of order → swap

1	4	2	5	8
---	---	---	---	---

Step 4 — compare 5 & 8 → already in order → no swap. End of Pass 1:

1	4	2	5	8
---	---	---	---	---

8 is now in its final correct position (green = sorted/locked).

	being compared		locked in final place
--	----------------	--	-----------------------

Algorithm

1. Compare each pair of adjacent elements from the start of the (unsorted portion of the) array.
2. If the left element is bigger than the right, swap them.
3. Continue to the end of the unsorted portion.
4. After each pass, the largest remaining value reaches the end.
5. Repeat until all elements are sorted.

Pseudocode

```
for i = 0 to n-2:           // n-1 passes
  for j = 0 to n-2-i:       // shrinks by 1 each pass
    if arr[j] > arr[j+1]:
      swap(arr[j], arr[j+1])
```

Key points to remember

- Stable — equal elements never cross each other (a swap only happens on strict '>').
- In-place — use only $O(1)$ extra space.
- Worst & average case: $O(n^2)$ — happens on a reverse-sorted array.
- Best-known weakness: a small value near the end ("turtle") moves toward the front very slowly, taking many passes. **Comb Sort fixes this by using larger gaps.**

2. Modified Bubble Sort (Optimized)

Idea: Add a "swapped" flag. If a full pass makes zero swaps, the array is already sorted — stop early instead of running all $n-1$ passes blindly.

Visual: Already-sorted input [1, 2, 3, 4, 5]

1	2	3	4	5
---	---	---	---	---

Pass 1 runs, compares every adjacent pair, finds nothing to swap → swapped stays false → algorithm exits immediately. Plain Bubble Sort would still blindly run 3 more passes here.

Algorithm

1. Do exactly what standard Bubble Sort do, but before each pass, set a flag: swapped = false.
2. Whenever a swap actually happens inside that pass, set swapped = true.
3. At the end of the pass, check the flag: if swapped is still false, no swaps occurred, meaning the array is already sorted.
4. If the array is sorted, break out of the loop immediately instead of running the remaining passes.

Pseudocode

```
for i = 0 to n-2:
  swapped = false           // reset every pass
  for j = 0 to n-2-i:
    if arr[j] > arr[j+1]:
      swap(arr[j], arr[j+1])
      swapped = true
  if swapped == false:
    break                   // array already sorted, stop early
```

Key points to remember

- This is the standard answer to "how would you optimize Bubble Sort?" in interviews.
- Best case improves to $O(n)$ — when the array is already sorted or nearly sorted. Worst/average case is still $O(n^2)$.
- Common bug: resetting swapped outside the outer loop (only once) instead of at the start of every pass.
- Still stable and in-place, uses $O(1)$ extra space.

3. Selection Sort

Idea: Split the array into sorted (left) and unsorted (right) parts. Repeatedly find the minimum of the unsorted part and swap it to the front of that part.

Visual: One Pass on [29, 10, 14, 37, 13]

29	10	14	37	13
----	----	----	----	----

Step 1 — assume index 0 (29) is the minimum so far (purple = current min).

29	10	14	37	13
----	----	----	----	----

Step 2 — compare min (29) with index 1 (10): $10 < 29 \rightarrow$ new min is index 1 (10).

29	10	14	37	13
----	----	----	----	----

Step 3 — compare min (10) with index 2 (14): not smaller $\rightarrow$ min stays index 1 (10).

29	10	14	37	13
----	----	----	----	----

Step 4 — compare min (10) with index 3 (37): not smaller $\rightarrow$ min stays index 1 (10).

29	10	14	37	13
----	----	----	----	----

Step 5 — compare min (10) with index 4 (13): not smaller $\rightarrow$ min stays index 1 (10). Scan of the unsorted part is complete.

10	29	14	37	13
----	----	----	----	----

Step 6 — swap index 0 and index 1. 10 is now locked in its final position — that's the whole pass.

	current minimum		being compared		locked in final place
--	-----------------	--	----------------	--	-----------------------

Algorithm

1. Assume the first element of the unsorted part (index i) is the minimum — set $\text{minIndex} = i$.
2. Scan the rest of the unsorted part (index $i+1$ to $n-1$); whenever a smaller element is found, update minIndex to that position.
3. Once the scan reaches the end, swap $\text{arr}[i]$ with $\text{arr}[\text{minIndex}]$ — this places the true minimum at the front of the unsorted part.
4. Move the unsorted boundary one step forward ($i = i+1$) and repeat until only one element remains.

Pseudocode

```
for i = 0 to n-2:
    minIndex = i
    for j = i+1 to n-1:
        if arr[j] < arr[minIndex]:
            minIndex = j
    swap(arr[i], arr[minIndex])
```

Key points to remember

- Always $O(n^2)$ comparisons — best, average, and worst case are identical, since it checks the remaining elements every time, even if the array is already sorted.
- At most $(n-1)$ swaps total — far fewer than Bubble Sort. Pick Selection Sort when writes/swaps are expensive (e.g., flash memory).
- NOT stable by default — a single long-distance swap can jump an element past an equal one. Example: [4A, 2, 4B, 1] $\rightarrow$ moving 1 to the front can make 4B come before 4A.
- In-place, $O(1)$ space.

4. Insertion Sort

Idea: Build a sorted section one element at a time by inserting each new element into its correct position among the already-sorted elements (like sorting cards in your hand).

Visual: Inserting 2 into [1, 3, 5, 8 | 2]

1	3	5	8	2
---	---	---	---	---

Step 1 — key = 2 (index 4, yellow). Compare with arr[3]=8: $8 > 2 \rightarrow$ shift 8 one step right.

1	3	5	8	8
---	---	---	---	---

Step 2 — hole (purple) now at index 3. Compare with arr[2]=5: $5 > 2 \rightarrow$ shift 5 one step right.

1	3	5	5	8
---	---	---	---	---

Step 3 — hole now at index 2. Compare with arr[1]=3: $3 > 2 \rightarrow$ shift 3 one step right.

1	3	3	5	8
---	---	---	---	---

Step 4 — hole now at index 1. Compare with arr[0]=1: $1 < 2 \rightarrow$ stop shifting.

1	2	3	5	8
---	---	---	---	---

Step 5 — insert key (2) into the hole at index 1. Array is now fully sorted.

key element	empty hole	just shifted	final position
-------------	------------	--------------	----------------

Algorithm

1. Treat the first element (index 0) as a sorted sub-array of size 1.
2. Take the next element (the "key") from the unsorted part.
3. Compare the key with elements in the sorted part, moving from right to left; shift every element greater than the key one position to the right.
4. Stop shifting as soon as you find an element smaller than (or equal to) the key, or you reach the start of the array.
5. Insert the key into the empty slot left behind. Repeat for every remaining element.

Pseudocode

```
for i = 1 to n-1:
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]    // shift right
        j = j - 1
    arr[j+1] = key           // insert key
```

Key points to remember

- Best case $O(n)$ on already-sorted input — the while loop never runs.
- Excellent for small or nearly-sorted arrays — that's why algorithms like **Timsort** and **Introsort** often use Insertion Sort for small parts of the array.
- "Online" algorithm — can sort data one element at a time as new data arrives, without needing the complete array first
- Stable, in-place, uses $O(1)$ extra space.

5. Shell Sort

Idea: A generalization of Insertion Sort. Compare elements that are 'gap' positions apart (not just adjacent), shrinking the gap each round down to 1 — the final gap=1 round is just a cheap Insertion Sort on an almost-sorted array.

Visual: [9, 8, 3, 7, 5, 6, 4, 1], starting gap = 4

9	8	3	7	5	6	4	1
---	---	---	---	---	---	---	---

Step 1 — gap=4: compare index 0 & 4: 9 vs 5 → out of order → swap.

5	8	3	7	9	6	4	1
---	---	---	---	---	---	---	---

Step 2 — compare index 1 & 5: 8 vs 6 → out of order → swap.

5	6	3	7	9	8	4	1
---	---	---	---	---	---	---	---

Step 3 — compare index 2 & 6: 3 vs 4 → already in order → no swap.

5	6	3	7	9	8	4	1
---	---	---	---	---	---	---	---

Step 4 — compare index 3 & 7: 7 vs 1 → out of order → swap. End of gap=4 pass:

5	6	3	1	9	8	4	7
---	---	---	---	---	---	---	---

gap = $4/2 = 2$. Compare elements 2 apart — this behaves like running Insertion Sort separately on the two interleaved sub-sequences (indices 0,2,4,6 and indices 1,3,5,7), then merging back:

3	1	4	6	5	7	9	8
---	---	---	---	---	---	---	---

gap = $2/2 = 1$. The final round is a plain Insertion Sort pass (same mechanics as Section 4) on this almost-sorted array — cheap because so few elements are out of place:

1	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---

Array is now fully sorted. Notice how the large first gap moved far-apart elements quickly, so the final gap=1 cleanup had very little work left to do.

Algorithm

1. Choose a starting gap (commonly $n/2$).
2. Perform a "gapped" insertion sort: for each element, compare it with the element 'gap' positions before it, shifting and inserting exactly like normal Insertion Sort, just at a distance instead of adjacent.
3. Once all elements have been processed at this gap, reduce the gap (commonly gap = gap / 2).
4. Repeat steps 2–3 with the smaller gap, continuing until gap = 1 — the final round is a plain Insertion Sort pass, but on an array, that's already nearly sorted.

Pseudocode

```
gap = n / 2
while gap > 0:
    for i = gap to n-1:           // gapped insertion sort
        key = arr[i]
        j = i
        while j >= gap and arr[j-gap] > key:
            arr[j] = arr[j-gap]
            j = j - gap
        arr[j] = key
    gap = gap / 2
```

Key points to remember

- Fixes plain Insertion Sort's slow "one step at a time" shifting by moving far-apart elements quickly first.
- Not stable — elements with the same value may change their original order because of gap-based comparisons. (e.g. 2A and 2B may become 2B, 2A.)
- Performance depends heavily on the gap sequence: simple $n/2$ halving can have $O(n^2)$ worst case, while Knuth's sequence (1, 4, 13, 40...) often performs better.
- In-place, $O(1)$ space. Good for medium-sized arrays where full $O(n \log n)$ sorting algorithm is unnecessary.

6. Comb Sort

Idea: Bubble Sort's direct upgrade. Start comparisons at a large gap (not just adjacent) and shrink the gap by a factor of ~ 1.3 each pass, ending with gap = 1 (a final bubble-sort-style cleanup).

Visual: Full Trace of [8, 4, 1, 56, 3, 44]

8	4	1	56	3	44
---	---	---	----	---	----

Start. $n=6$, $gap = 6/1.3 \approx 4$.

8	4	1	56	3	44
---	---	---	----	---	----

Step 1 — $gap=4$: compare index 0 & 4: 8 vs 3 $\rightarrow$ swap.

3	4	1	56	8	44
---	---	---	----	---	----

Step 2 — compare index 1 & 5: 4 vs 44 $\rightarrow$ already in order, no swap. End of $gap=4$ pass. New $gap = 4/1.3 \approx 3$.

3	4	1	56	8	44
---	---	---	----	---	----

$gap=3$: compare (0,3)=3&56, (1,4)=4&8, (2,5)=1&44 — all already in order, so NOTHING changes this round. (Good exam point: a gap round can do zero swaps and still not be redundant — it's setting up for smaller gaps.) New $gap = 3/1.3 \approx 2$.

3	4	1	56	8	44
---	---	---	----	---	----

Step 3 — $gap=2$: compare index 0 & 2: 3 vs 1 $\rightarrow$ swap.

1	4	3	56	8	44
---	---	---	----	---	----

Step 4 — compare index 1 & 3: 4 vs 56 $\rightarrow$ no swap.

1	4	3	56	8	44
---	---	---	----	---	----

Step 5 — compare index 2 & 4: 3 vs 8 $\rightarrow$ no swap.

1	4	3	56	8	44
---	---	---	----	---	----

Step 6 — compare index 3 & 5: 56 vs 44 $\rightarrow$ swap. End of $gap=2$ pass. New $gap = 2/1.3 \approx 1$.

1	4	3	44	8	56
---	---	---	----	---	----

$gap=1$: from here Comb Sort behaves exactly like Bubble Sort (adjacent comparisons only) — it keeps making full passes until one pass makes zero swaps (see Section 1 for that mechanic in detail).

1	3	4	8	44	56
---	---	---	---	----	----

Final result after the $gap=1$ pass finish. Notice the turtle-style long-distance moves happened early (large gaps) — that's exactly what plain Bubble Sort lacks.

Algorithm

1. Initialize gap = n (array length) and set shrink factor = 1.3.

2. Update the gap each round: $\text{gap} = \text{gap} / 1.3$ (rounded down to an integer). If this makes $\text{gap} < 1$, set $\text{gap} = 1$.
3. Compare $\text{arr}[i]$ with $\text{arr}[i+\text{gap}]$ for every valid i ; swap if out of order, and set a swapped flag to true whenever a swap happens.
4. Repeat steps 2–3 until gap equals 1 AND a full pass completes with zero swaps — that's when the array is fully sorted.

Pseudocode

```
gap = n
shrink = 1.3
swapped = true
while gap != 1 or swapped == true:
    gap = int(gap / shrink)
    if gap < 1: gap = 1
    swapped = false
    for i = 0 to n-gap-1:
        if arr[i] > arr[i+gap]:
            swap(arr[i], arr[i+gap])
            swapped = true
```

Key points to remember

- Directly solves **Bubble Sort's "turtle" problem** — small values near the end can move forward much faster instead of crawling one position at a time.
- **Why 1.3?** — Testing showed that 1.3 works well as the gap-shrinking factor. It was found through experiments, not a mathematical formula. This is a common question.
- Not stable, in-place, uses $O(1)$ space.
- If gap is forced to always be 1, Comb Sort literally becomes Bubble Sort.

High-Yield Facts for Exams

- **Stability:** Bubble, Modified Bubble, Insertion → stable. Selection, Shell, Comb → not stable.
- **Best case $O(n)$:** Modified Bubble Sort and Insertion Sort, when the array is already/nearly sorted.
- **Fewest swaps:** Selection Sort (max $n-1$ swaps) — use when swap/write cost is high.
- **Best for small/nearly-sorted data:** Insertion Sort — also the only "online" sort in this list.
- **Gap-based generalizations:** Shell Sort = Insertion Sort + shrinking gap. Comb Sort = Bubble Sort + shrinking gap (shrink factor 1.3).
- **All six:** In-place, $O(1)$ auxiliary space, and $O(n^2)$ in the worst case (Shell/Comb can do better with a good gap sequence).
- **Loop bounds:** $n-1$, $n-2$, or $i < n-1$ are all the same idea in different notations. Understand **why** the loop stops where it does instead of memorizing numbers.